

PORTADA

Calidad de Software

Semana 5 — Deuda técnica, buenas prácticas y herramientas de apoyo

Universidad de Antioquía · Ingeniería de Sistemas



ENCUADRE

Última semana de grupos aleatorios

Desde la semana 6 se trabaja en los **equipos fijos de Fábrica-Escuela** — el mismo equipo hasta el final del semestre. Dos cosas cierran hoy la Unidad 2:

- El último taller con grupo aleatorio de la semana.
- La **entrega 1 de Fábrica-Escuela** (10 % de la nota), con los temas propios del proyecto de cada equipo.



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 4

- **ISO 9000** es gestión de calidad en general; **SQuaRE** es la familia específica de software donde vive ISO/IEC 25010.
- Las revisiones van de informales a formales: **walkthrough, revisión técnica, inspección, auditoría.**
- Cada hallazgo se documenta con **severidad** y la característica de calidad comprometida.



TRANSICIÓN

Hoy: de las normas a la práctica diaria

Las últimas cuatro semanas hablaron de normas, procesos y revisiones formales. Hoy bajamos al terreno del día a día: el código que se escribe entre una revisión y la siguiente, y cómo se degrada silenciosamente si nadie lo cuida.

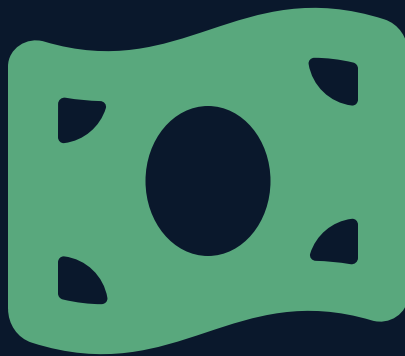


CONCEPTO · ORIGEN

Qué es la deuda técnica

El término lo acuñó **Ward Cunningham** en 1992 —el mismo Cunningham que años después crearía el primer wiki— como metáfora para explicarle a un cliente por qué el equipo necesitaba tiempo para reescribir código.

La idea: escribir código rápido y no del todo limpio es como pedir un préstamo. Se puede hacer, y a veces es la decisión correcta — pero la deuda genera **interés**, y ese interés se paga en esfuerzo extra cada vez que alguien toca ese código después.



CONCEPTO

El cuadrante de Martin Fowler

Fowler propuso que no toda deuda técnica es un error — depende de si fue una decisión consciente y de si fue razonable tomarla:

	Deliberada	Inadvertida
Prudente	"Sabemos que esto no es ideal, pero entregamos ya y lo arreglamos después."	"Ahora que lo terminamos, vemos cómo debimos haberlo hecho."
Imprudente	"No tenemos tiempo para hacer las cosas bien."	"¿Qué es diseño de capas?"

La deuda prudente y deliberada, bien documentada, es una herramienta legítima — no siempre es una falla.

CONCEPTO

Cómo se acumula la deuda técnica

- **Plazos apretados:** se prioriza entregar sobre hacerlo bien.
- **Falta de pruebas:** nadie se atreve a refactorizar sin una red de seguridad.
- **Requisitos que cambian:** el diseño original ya no encaja, pero nadie lo actualiza.
- **Rotación del equipo:** se pierde el contexto de por qué el código quedó así.
- **Código copiado y pegado** en vez de reutilizado: el mismo defecto se multiplica.



CONCEPTO

El interés de la deuda técnica

Igual que una deuda financiera, cuanto más tiempo pasa sin pagarse, **más caro sale cada cambio nuevo**: hay que entender código enredado, rodear el problema en vez de resolverlo, y el riesgo de romper algo al tocarlo crece.

La trampa común: como el interés no se ve en un balance contable, es fácil ignorarlo hasta que un cambio simple —que debería tomar una hora— termina tomando una semana.



TRANSICIÓN

Cómo se detecta la deuda técnica

La deuda técnica no siempre se ve a simple vista en la funcionalidad —el sistema puede seguir "funcionando"—, pero sí deja señales reconocibles en la estructura del código.



CONCEPTO

Qué es un code smell

El término lo popularizaron Kent Beck y Martin Fowler en el libro *"Refactoring"* (1999): un **"olor a código"** es una señal superficial —no un error en sí mismo— que suele indicar un problema más profundo de diseño.

- **Método largo:** hace demasiadas cosas para entenderlo de un vistazo.
- **Código duplicado:** la misma lógica repetida en varios lugares.
- **Clase gigante:** acumula responsabilidades que deberían estar separadas.
- **Lista larga de parámetros:** señal de que faltan abstracciones.



CONCEPTO

Qué es refactorizar

Fowler lo define con precisión: **cambiar la estructura interna de un código sin cambiar su comportamiento observable**. Es la manera en que se paga la deuda técnica.

- **No es lo mismo que reescribir** — reescribir empieza de cero; refactorizar transforma en pasos pequeños y verificables.
- Necesita pruebas automatizadas que confirmen, en cada paso, que el comportamiento no cambió.
- Se hace en pasos pequeños, no en una sola operación gigante y arriesgada.



CONCEPTO

Buenas prácticas para no acumular deuda

- **Nombres significativos:** una variable llamada `deuda` es deuda técnica el día en que se escribe.
- **Funciones pequeñas, una responsabilidad cada una:** más fáciles de leer, probar y reutilizar.
- **No te repitas (DRY):** cada pieza de lógica debería vivir en un solo lugar.
- **Comentarios solo cuando hacen falta:** si el código necesita un comentario para explicarse, primero considera si se puede reescribir para que no lo necesite.

TRANSICIÓN

Cómo se mide todo esto automáticamente

Encontrar code smells leyendo código a mano no escala a un repositorio con miles de líneas. Existe una categoría de herramientas que lo hace por nosotros: el **análisis estático**.



CONCEPTO

Qué es el análisis estático

Es examinar el código **sin ejecutarlo** —a diferencia de una prueba, que sí lo ejecuta— buscando patrones que suelen indicar bugs, vulnerabilidades o code smells. En el vocabulario de la semana 2, es una técnica de **verificación**: compara el código contra reglas, no contra la necesidad del usuario.

Ya se mencionó de pasada en la semana 2 como una de las técnicas de verificación — hoy se ve en detalle, con una herramienta real.



CONCEPTO · HERRAMIENTA

Qué es SonarCloud

SonarCloud es la versión en la nube de SonarQube, la herramienta de análisis estático de SonarSource. Se conecta directamente al repositorio (GitHub, GitLab) y analiza el código en cada commit o pull request.

Es **gratuita para proyectos de código abierto** — por eso es la herramienta que se usa en este curso y la que van a integrar a su pipeline de Fábrica-Escuela más adelante.



CONCEPTO

Qué mide SonarCloud

Bugs	Código que probablemente se comporta de forma distinta a la esperada
Vulnerabilidades	Puntos débiles explotables de seguridad
Code Smells	Señales de mantenibilidad — los mismos que vimos hace unas diapositivas
Cobertura	Porcentaje del código ejercitado por pruebas automatizadas
Duplicación	Porcentaje de código repetido en el proyecto
Complejidad ciclomática	Qué tan enredado es el flujo de un método — se profundiza en la semana 11

CONCEPTO

El Quality Gate

Es un conjunto de condiciones —definidas por el equipo— que un análisis debe cumplir para considerarse **aprobado**: por ejemplo, "cero bugs nuevos", "cobertura mayor al 80 % en código nuevo", "cero vulnerabilidades críticas".

Si el análisis no cumple el Quality Gate, queda en **rojo** — una señal visual e inmediata de que algo no está listo para fusionarse. Es la pieza que, más adelante en el curso, se conecta con el pipeline de integración continua.



CONCEPTO

La severidad en SonarCloud

SonarCloud usa su propia escala, distinta —pero comparable— a la Crítica/Alta/Media/Baja que vimos en la semana 4:

Blocker	Debe corregirse antes de continuar — riesgo muy alto de bug o vulnerabilidad
Critical	Impacto grave en el comportamiento o la seguridad
Major	Afecta la mantenibilidad de forma importante
Minor / Info	Mejoras menores, sin urgencia

La herramienta asigna la severidad automáticamente — priorizar sigue siendo trabajo humano.

CONCEPTO

Qué es un plan de remediación

Encontrar un hallazgo no es suficiente: hay que decidir **qué hacer con él**. Un plan de remediación, por cada hallazgo priorizado, responde:

- **Qué cambio concreto** corrige el problema.
- **Cuánto esfuerzo estimado** toma —en horas, o en una escala relativa (pequeño/mediano/grande).
- **Cuándo se aborda** — no todo se corrige de inmediato; a veces se documenta como deuda deliberada y se agenda para después.



EJEMPLO

Un hallazgo con plan de remediación

Hallazgo	Método <code>procesarPago()</code> con complejidad ciclomática de 24 (18 anidamientos de <code>if</code>)
Severidad	Mayor
Por qué importa	Casi imposible de probar exhaustivamente; alto riesgo al modificarlo
Remediación	Extraer cada rama en un método propio; considerar un patrón de estrategia
Esfuerzo estimado	Mediano (~6 horas, con pruebas de regresión antes y después)

INTERACCIÓN

Ejercicio rápido: encuentren el code smell

```
def procesar(a, b, c, d, e, f, tipo, activo, fecha, extra):  
    if tipo == 1:  
        if activo:  
            if fecha is not None:  
                # ... 40 líneas más de lógica anidada
```

5 minutos: ¿qué code smell(s) identifican? ¿Cómo empezarían a refactorizarlo?

Este ejercicio es el calentamiento del taller de hoy.

CONCEPTO EN DISPUTA

¿La herramienta reemplaza el juicio humano?

Es tentador tratar "Quality Gate en verde" como sinónimo de "código de calidad". No lo es: SonarCloud detecta **patrones**, no intención — produce falsos positivos, y un código puede tener 0 % de hallazgos y aun así resolver mal el problema del usuario.

El riesgo simétrico es "perseguir el número": subir cobertura con pruebas triviales que no verifican nada real, solo para que el porcentaje se vea bien.

Las métricas son una brújula, no un mapa: señalan dirección, no reemplazan la revisión humana que vimos la semana pasada.



SÍNTESIS

Lo que hay que llevarse de hoy

1. La **deuda técnica** no siempre es un error — depende de si fue deliberada y prudente.
2. Un **code smell** no es un bug: es una señal de un problema de diseño más profundo.
3. **Refactorizar** cambia la estructura, no el comportamiento — y necesita pruebas para hacerse con seguridad.
4. El **análisis estático** examina código sin ejecutarlo; el **Quality Gate** define cuándo un análisis se considera aprobado.
5. Priorizar y estimar el esfuerzo de remediar sigue siendo **trabajo humano**, no algo que la herramienta decide sola.



INSTRUCCIONES

Taller de hoy: análisis con SonarCloud

2 horas, en sala de Zoom asignada — grupo aleatorio de la semana

El docente asigna un repositorio distinto a cada equipo. Cada equipo debe analizarlo con SonarCloud y entregar:

- **Tres hallazgos priorizados**, cada uno con su severidad reportada por la herramienta.
- Un **plan de remediación** por hallazgo: qué cambio concreto lo corrige.
- Una **estimación de esfuerzo** por hallazgo.

La entrega cierra al terminar las dos horas. Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 6, hay que llegar con esto leído:

- Plan de pruebas: alcance, estrategia, criterios de entrada y salida, riesgos, recursos.

Empieza la **Unidad 4 — Fundamentación en pruebas**, con los equipos fijos de **Fábrica-Escuela** desde hoy hasta el final del semestre. El taller de la semana 6: plan de pruebas de una página para ParaBank.

